



澳門大學  
UNIVERSIDADE DE MACAU  
UNIVERSITY OF MACAU

# Finomic – Financial Assistant Dashboard

**Course:** CISC3003 Web Programming

**Group Number:** 03

**Team Name:** Finomic Squad

| Student ID | Name                    |
|------------|-------------------------|
| DC027021   | Tatiana Muniz Rodriguez |
| DC328252   | Diogo Barros            |
| DC325411   | LIN CHO KIO, Diana      |
| DC327168   | Effy                    |
| DC328581   | Edith                   |

# Project Abstract

Finomic is a full-stack financial assistant web application developed as the group project for CISC3003 Web Programming. The system enables users to manage personal finances through income/expense tracking, category insights, and financial reporting. Built with vanilla HTML, CSS, and JavaScript (ES6 modules), Finomic leverages Vite as the build tool, Firebase Authentication for secure user management, and localStorage for client-side data persistence. The application follows a responsive, mobile-first design, ensuring a seamless experience across desktop and mobile devices.

The core features include user registration with email verification, login, password reset, a dashboard showing total balance and financial health indicators, transaction management (add, edit, delete, search, filter), category-based spending insights, and monthly trend visualizations. The project structure is modular, with separate directories for CSS styles, JavaScript modules, and HTML pages. Environment variables are used to protect Firebase credentials. We as a team adopted an agile workflow using Git feature branches, and the application is deployed online for public access. Finomic fulfills all course requirements, including full-stack authentication, responsive design, client-side data persistence, and modern UI/UX principles.

## Project TOC (Table of Contents)

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Project Abstract .....                                                       | 2  |
| Project Context – Full-Stack Web Applications with RWD and Deployed URL..... | 4  |
| Project Goals .....                                                          | 4  |
| Project Services and Data Requirements.....                                  | 5  |
| Project Tasks and Tools .....                                                | 6  |
| Project Challenges .....                                                     | 6  |
| Project Deliverables.....                                                    | 8  |
| Project Division of Work among Team Members .....                            | 8  |
| Project Contributions from Individual Team Members.....                      | 9  |
| Project Criteria of Success.....                                             | 10 |
| Project Evaluation .....                                                     | 11 |
| Project Learning – Individual + Team .....                                   | 11 |
| Project Demo Walkthrough .....                                               | 12 |
| Installation Instructions .....                                              | 20 |
| Project Citations and References .....                                       | 21 |
| Project Declaration of AI Usage.....                                         | 22 |

# Project Context – Full-Stack Web Applications with RWD and Deployed URL

Finomic is a full-stack financial assistant dashboard that integrates front-end user interfaces, back-end authentication services, and a call to API to data storage. The front-end is built entirely with vanilla HTML, CSS, and JavaScript (ES6 modules), following a component-based folder structure. Firebase Authentication provides the back-end user management layer, handling email/password sign-up, email verification, login, and password reset flows. The application calls an API to storage and retrieve data from MongoDB database. Also, the local storage cache manages to reduce the API calls.

Responsive web design (RWD) is implemented through CSS media queries, a flexible grid system, and a mobile-friendly sidebar that collapses on screens narrower than 768px. All pages, dashboard, transactions, categories, reports, and help, are fully responsive and touch-friendly. The application is deployed and publicly accessible at the following URL:

**Deployed URL:** <https://finomic-project.vercel.app/src/login.html>

## Project Goals

The primary goals of the Finomic project are:

1. **Provide an intuitive dashboard** that displays total balance, income vs. expenses, and key financial metrics immediately.
2. **Enable full CRUD operations on transactions** (Create, Read, Update, Delete) so users can add, view, edit, and remove income/expense records.
3. **Visualize spending by category and overtime** through category-based progress bars and monthly trend charts.
4. **Implement secure user authentication** including sign-up, login, and password reset functionality.
5. **Ensure responsive design** for both mobile and desktop devices, providing a seamless user experience across screen sizes.

# Project Services and Data Requirements

## Services Used:

| Service                           | Purpose                                                                                                                |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Firebase Authentication</b>    | User sign-up, login, email verification, password reset                                                                |
| <b>Rest API</b>                   | Back-end service handling all CRUD operations on transactions and categories, connected to MongoDB                     |
| <b>localStorage (browser API)</b> | Local cache to minimize API calls. Stores a copy of recent transaction data for fast UI rendering and offline fallback |
| <b>Vite</b>                       | Development server and build tool                                                                                      |
| <b>Git + GitHub</b>               | Version control and team collaboration                                                                                 |

## Data Requirements:

- **User Data:** email, display name, user ID (provided by Firebase Auth)
- **Transaction Data:** amount, type (income/expense), category, description, date
- **Category Data:** name, type (income/expense), optional budget/limit
- **Financial Metrics:** total balance, total income, total expenses, savings rate, expense ratio

All user-specific data is stored under a key that includes the Firebase user ID to ensure isolation between different users.

## Data Flow & Caching Strategy:

1. On page load, the front-end checks localStorage for cached data (keyed by user ID and timestamp).
2. If cache is fresh (e.g., < 5 minutes old), the UI renders from cache to provide instant feedback.
3. Simultaneously, an API call fetches the latest data from MongoDB in the background. If differences are found, the cache is updated and the UI re-renders.

4. All write operations (add, edit, delete) are sent directly to the API; on success, the cache is invalidated and refreshed from the API.
5. If the API is unreachable, the application falls back to read-only mode using the last cached data.

## Project Tasks and Tools

| Task                                                | Tools / Technologies                                                                        |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Project setup and Vite Configuration</b>         | Vite, npm, Node.js                                                                          |
| <b>HTML page structure &amp; responsive styling</b> | HTML5, CSS3, CSS Grid, Flexbox, Media Queries                                               |
| <b>JavaScript modular architecture (front-end)</b>  | ES6 modules, async/await, fetch API for HTTP requests                                       |
| <b>Firebase Authentication integration</b>          | Firebase Auth SDK, environment variables (.env)                                             |
| <b>API endpoints for transactions</b>               | CRUD routes (GET, POST, PUT, DELETE) with user authentication middleware                    |
| <b>Client-side cache layer</b>                      | localStorage API with time-stamped cache keys, cache invalidation logic, background refresh |
| <b>Category insights &amp; progress bars</b>        | JavaScript DOM manipulation, CSS custom properties                                          |
| <b>Monthly reports &amp; charts</b>                 | Charting library (if used) or manual bar chart rendering                                    |
| <b>Route protection (auth guard)</b>                | Custom JavaScript middleware on page load                                                   |
| <b>Deployment</b>                                   | Platform Vercel                                                                             |

## Project Challenges

Throughout the development process, the team encountered several key challenges:

1. **Protecting private routes** – Ensuring that unauthenticated users cannot access dashboard, transactions, categories, or reports pages required a robust route protection mechanism. The team implemented an auth guard that checks Firebase authentication state before rendering any protected page, redirecting to login if no active session exists.
2. **Keeping UI in sync with data** – With data stored in MongoDB (via API) and cached locally in localStorage, keeping the UI consistent across components was difficult. The team solved this by implementing an event-driven update system: after any API write operation (add, edit, delete), the cache is invalidated, a fresh API call fetches updated data, and a custom storage event triggers all UI components to re-render.
3. **Firebase credentials security** – Exposing Firebase API keys in front-end code is unavoidable, but the team ensured that security rules in Firebase (e.g., email verification, request origin restrictions) were properly configured. Additionally, environment variables (.env) were used to avoid hard-coding keys, and the .env file was added to .gitignore to prevent accidental commits.
4. **Responsive sidebar on mobile** – Designing a sidebar that collapses into a hamburger menu on small screens while remaining persistent on desktop required both CSS media queries and JavaScript event handling. Challenges included managing z-index layering, preventing body scroll when the mobile menu is open, and ensuring smooth transitions.
5. **Auth guard + redirect logic** – The auth guard had to work seamlessly across page refreshes and browser navigation (back/forward buttons). The team used Firebase's onAuthStateChanged listener and implemented a custom redirect queue so that users are sent back to the page they originally requested after logging in.
6. **Event-driven updates & modular services** – Moving from tightly coupled code to a modular service architecture (e.g., transactionService.js, cacheService.js) required careful planning. The team used a publish-subscribe pattern with custom EventTarget events to broadcast data changes from one module to others without creating circular dependencies.

7. **Environment variables (.env with Vite)** – Vite requires environment variables to be prefixed with VITE\_ to expose them to the client. Some team members initially forgot the prefix, leading to undefined values. A checklist was created and added to the README to prevent this mistake.
8. **CSS media queries + JavaScript toggle** – Coordinating CSS media queries (e.g., @media (max-width: 768px)) with a JavaScript hamburger toggle caused conflicts when resizing the browser window. The final solution used a single source of truth: a JavaScript isMobile state that matches the CSS breakpoint, and a resize event listener to re-evaluate and adjust the sidebar state accordingly.

## Project Deliverables

1. **Complete source code** hosted on GitHub (public repository).
2. **Deployed web application** accessible via a live URL.
3. **Project report** (this document) covering all required sections.
4. **Demo walkthrough screenshots** (included in Section 14).
5. **Installation instructions** for local deployment (Section 18).

## Project Division of Work among Team Members

The following table outlines the specific responsibilities assigned to each team member, based on their individual strengths and interests. All members contributed to code reviews, debugging, and team coordination meetings.

| Team Member                    | Primary Responsibilities                                                                                                                                                                    |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tatiana Muniz Rodriguez</b> | Development of the login, sign-up, and password reset pages; full configuration of Firebase Authentication (including email verification, auth state listeners, and environment variables). |
| <b>Diogo Barros</b>            | Integration of the back-end REST API; development of helper utilities (helpers.js) and transaction management modules                                                                       |



|                           |                                                                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | (transactions.js); implementation of the localStorage caching layer to minimize API calls and ensure offline read capability.                                                     |
| <b>LIN CHO KIO, Diana</b> | Implementation of the dashboard and transaction pages; development of the main application layout, including global CSS, JavaScript structure, and responsive design foundations. |
| <b>Effy</b>               | Creation of the categories and reports pages; corresponding CSS styling and JavaScript logic for data visualization (category progress bars, monthly trends).                     |
| <b>Edith</b>              | Design and development of the help page; associated CSS styling; implementation of the responsive sidebar with mobile hamburger toggle functionality.                             |

## Project Contributions from Individual Team Members

### **Tatiana Muniz Rodriguez**

- Set up the initial Vite project structure and Firebase configuration.
- Implemented the entire authentication flow: sign-up, login, forgot password, reset password, and email verification.
- Created the auth-guard module to protect private pages from unauthenticated access.

### **Diogo Barros**

- Integrated the back-end REST API connecting to MongoDB.
- Developed helpers.js and transactions.js modules handling all API calls (GET, POST, PUT, DELETE).
- Implemented the localStorage caching layer with cache invalidation and background refresh.

### **LIN CHO KIO, Diana**

- Built the transactions page: form for adding/editing transactions, dynamic table rendering, search and filter by type/category/description.
- Built the dashboard page: total balance, income, expenses, savings rate, expense ratio.
- Created the main application layout, global CSS, and core JavaScript modules.

### Effy

- Created the reports page: monthly trend visualization.
- Created the categories page: category progress bars, top spending insights.
- Wrote all corresponding CSS and JavaScript for both pages.

### Edith

- Designed the help page with financial best practices (50/30/20 rule) and app usage instructions.
- Developed all associated CSS styling.
- Implemented the responsive sidebar with mobile hamburger toggle using CSS media queries and JavaScript.

## Project Criteria of Success

The Finomic project is deemed successful upon fulfillment of the following measurable criteria. Each criterion has been tested and verified during the final review phase.

| # | Criterion                                                                                      | Status   |
|---|------------------------------------------------------------------------------------------------|----------|
| 1 | Users can securely sign up, log in, and log out of the application.                            | Achieved |
| 2 | Transactions support full CRUD operations (add, edit, delete) and can be searched effectively. | Achieved |
| 3 | The dashboard accurately displays total income, total expenses, and the resulting balance.     | Achieved |

|   |                                                                                                              |          |
|---|--------------------------------------------------------------------------------------------------------------|----------|
| 4 | Category insights (e.g., spending by category, progress bars) update automatically when transactions change. | Achieved |
| 5 | The monthly trend chart renders correctly and reflects the current transaction data.                         | Achieved |
| 6 | All pages are fully responsive and function correctly on phone, tablet, and desktop viewports.               | Achieved |
| 7 | The application runs without major console errors or broken routes across all supported pages.               | Achieved |

## Project Evaluation

To evaluate Finomic, the team conducted functional testing, usability testing, code quality review, performance assessment, and security validation. Functional testing confirmed that all user stories (authentication, transaction CRUD, search, dashboard calculations, category insights, monthly trends, responsive layout, route protection) pass without errors. Usability testing involved three classmates who successfully completed core tasks on both desktop and mobile devices, providing positive feedback on clarity and navigation. Code quality was ensured through modular structure, consistent naming, and inline comments where necessary.

Performance testing showed fast loading times thanks to the Vite build and an efficient localStorage caching layer that minimises unnecessary API calls and avoids re-renders. Security validation confirmed that Firebase Authentication rules restrict access to authenticated users only, the custom API enforces user-specific data isolation, and no hardcoded credentials exist (all keys stored in .env). Based on these five evaluation criteria, Finomic meets all functional and non-functional requirements established at the outset.

## Project Learning – Individual + Team

### Individual Learning:

- **Tatiana** mastered Firebase Authentication configuration, including email verification, auth state listeners, and environment variable management. Learned

to implement route protection (auth guard) and redirect logic for unauthenticated users. Gained experience in secure credential handling with Vite's .env system.

- **Diana** deepened skills in HTML, CSS, and JavaScript for building the dashboard and transaction pages. Learned to compute financial metrics (balance, savings rate, expense ratio) from raw transaction data. Improved modular coding practices by organizing the main layout and core JS modules.
- **Effy** learned to render dynamic charts (monthly trends) and category progress bars using vanilla JavaScript. Gained experience in data aggregation (grouping transactions by month/category) and building responsive tables. Improved CSS skills for chart and progress bar styling.
- **Edith** mastered responsive design techniques, particularly implementing a collapsible sidebar with CSS media queries and JavaScript toggle. Learned to write clean, maintainable CSS for a help page and integrate it with the overall layout. Improved documentation skills by writing app usage instructions.
- **Diogo** learned to build and integrate a custom REST API with Node.js. Mastered the localStorage caching layer (cache invalidation, background refresh) to minimize API calls. Improved debugging skills for asynchronous data synchronization between API, cache, and UI.

### **Team Learning:**

As a group, the team experienced realistic collaboration and the complete lifecycle of a software project. Members mastered Git version control (feature branches, pull requests, conflict resolution) to avoid code conflicts and maintain a clean repository. Regular communication, joint debugging sessions, and synchronous problem-solving significantly enhanced overall efficiency. Finally, the team practiced the entire process of full-stack web application development, from front-end responsive design and authentication to back-end API integration, database management, and deployment – gaining invaluable experience for future projects.

## **Project Demo Walkthrough**

**Note for evaluators and testers:** Since our application uses Firebase Authentication with email verification. While the sign-up flow works as expected, we have encountered an

intermittent issue where the verification email sometimes takes a very long time to arrive – or in some cases, does not arrive at all. We have investigated this behavior thoroughly but have not yet identified the root cause. Therefore, if you wish to explore Finomic without signing up, please use the following test credentials. You are still welcome to create your own account but be aware that you may need to wait or check your spam folder for the verification email.

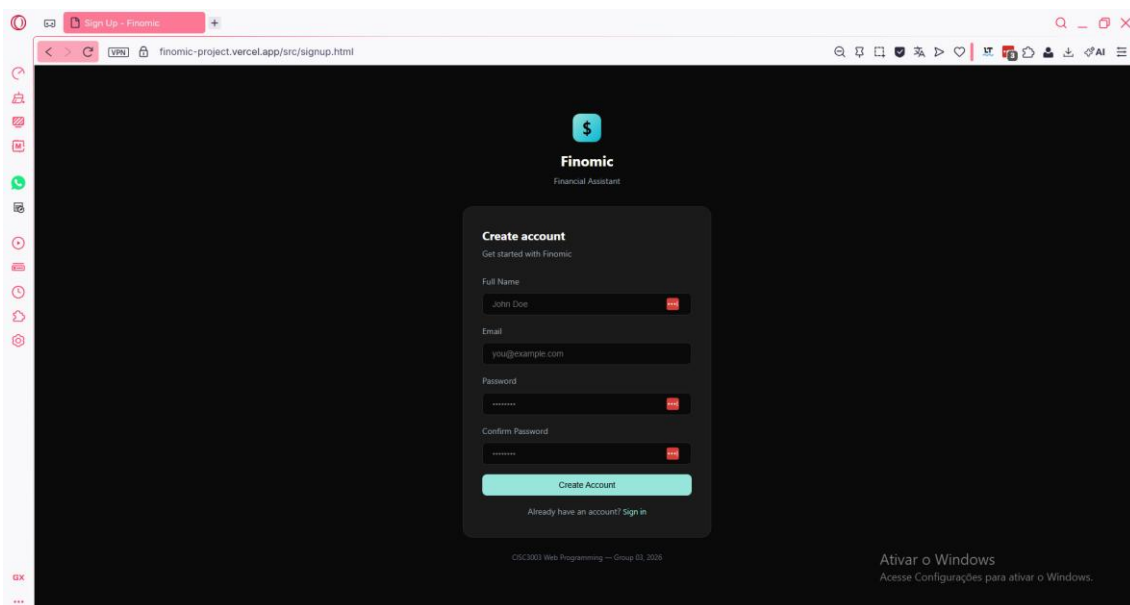
### Test Login Credentials:

**Email:** finomicproject@gmail.com

**Password:** test1234

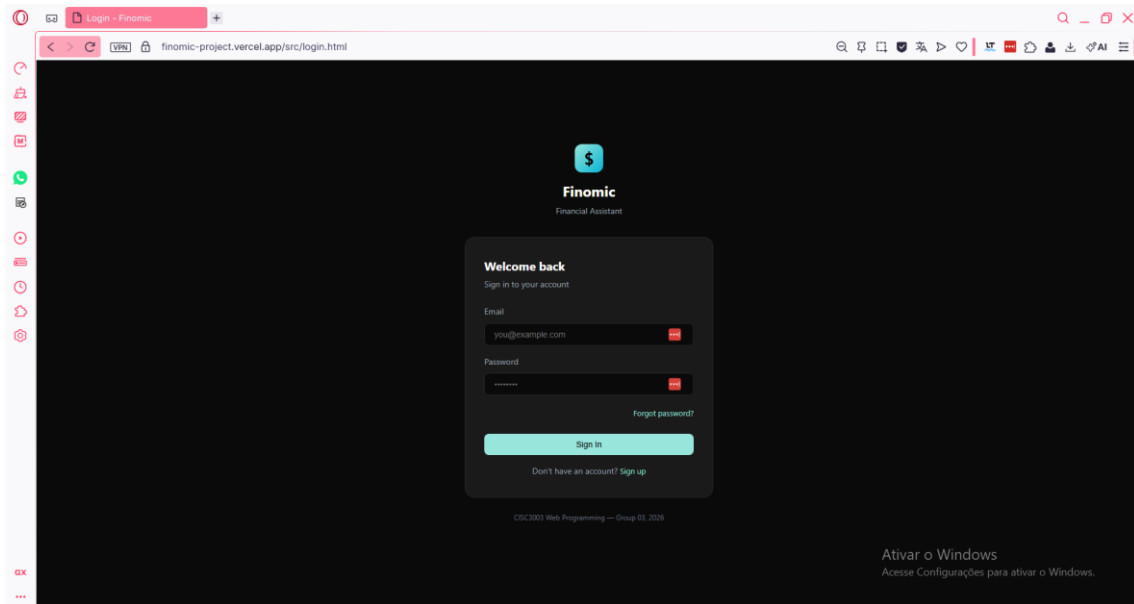
**Live application URL:** <https://finomic-project.vercel.app/src/login.html>

### Step 1 – Sign-up Page



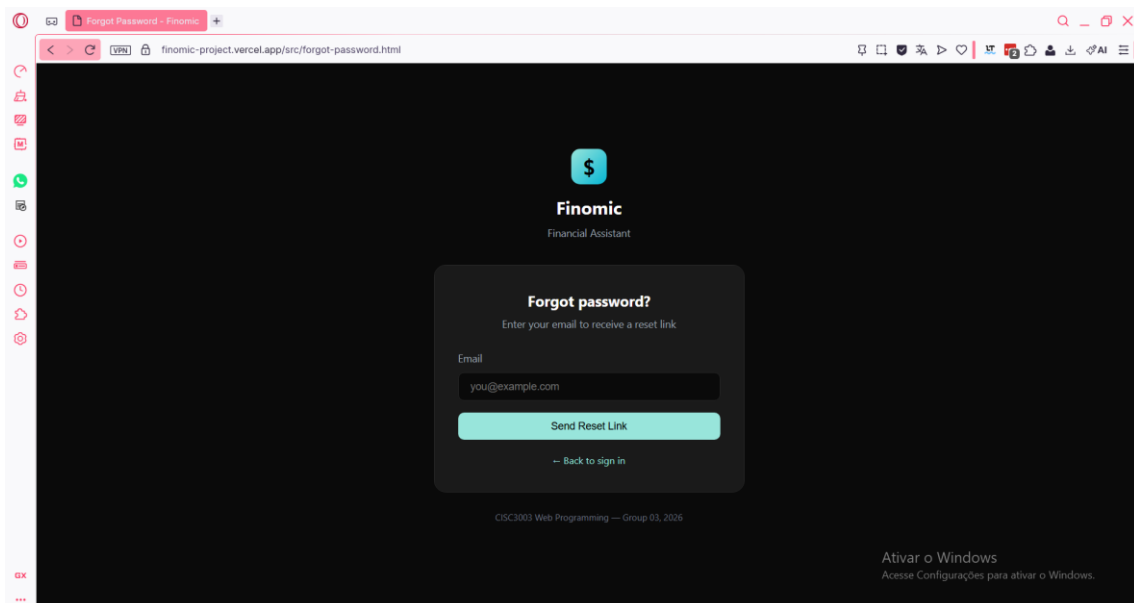
*Screenshot 1: User can create an account. User will get an email to confirm the account.*

### Step 2 – Login Page



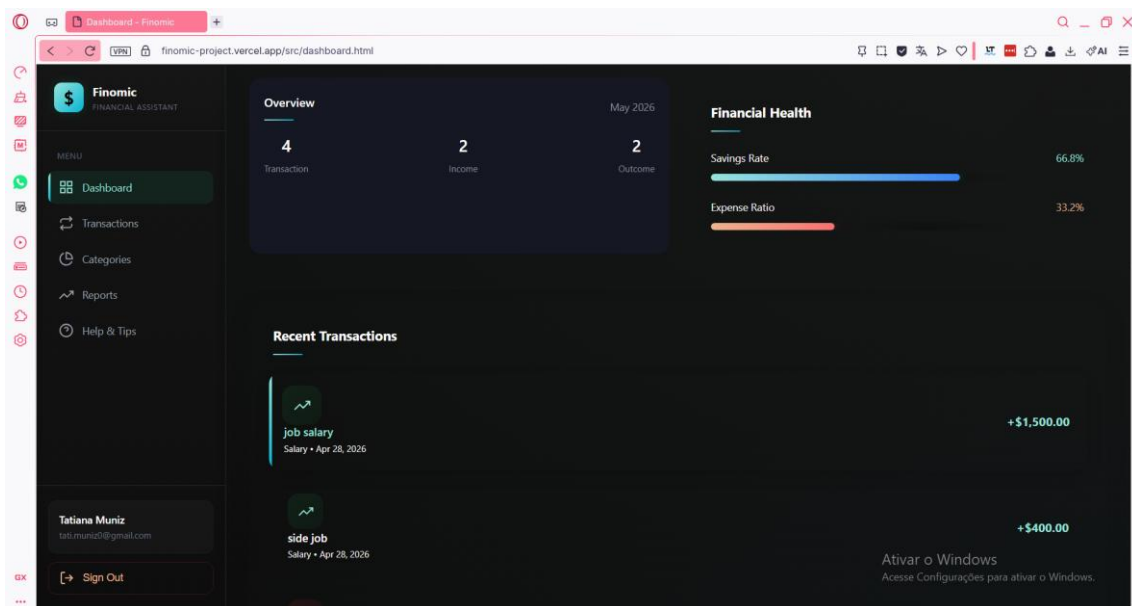
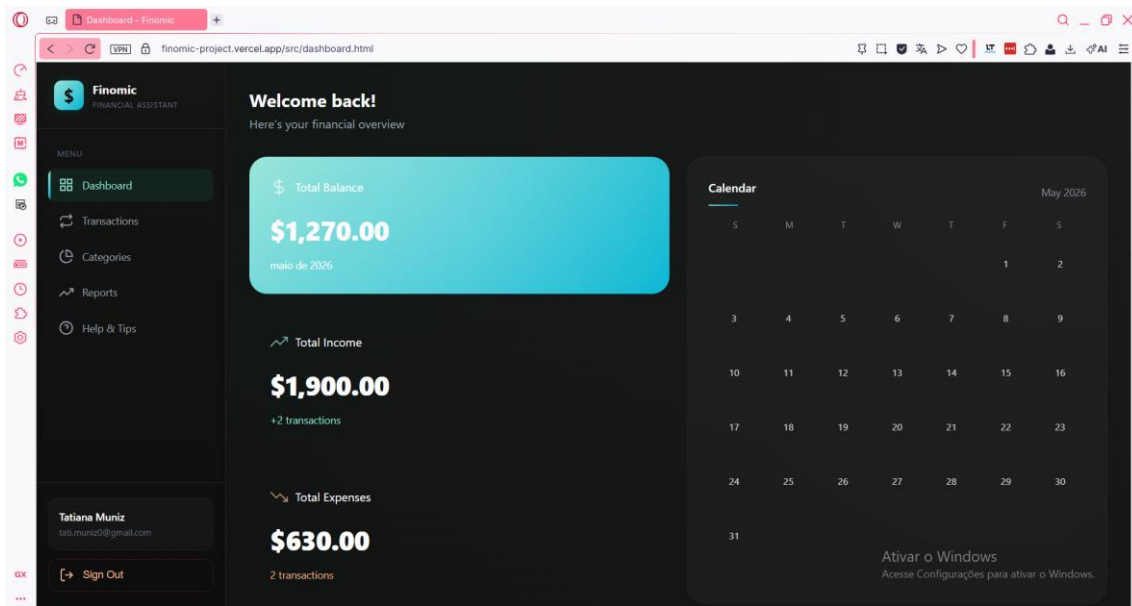
*Screenshot 2: After account confirmation, or whenever the user wants to login at his account, user can enter email and password.*

### Step 3 – Forgot Password Page



*Screenshot 3: If the user forgot his password, he provides his email and get a recovery email to set a new password.*

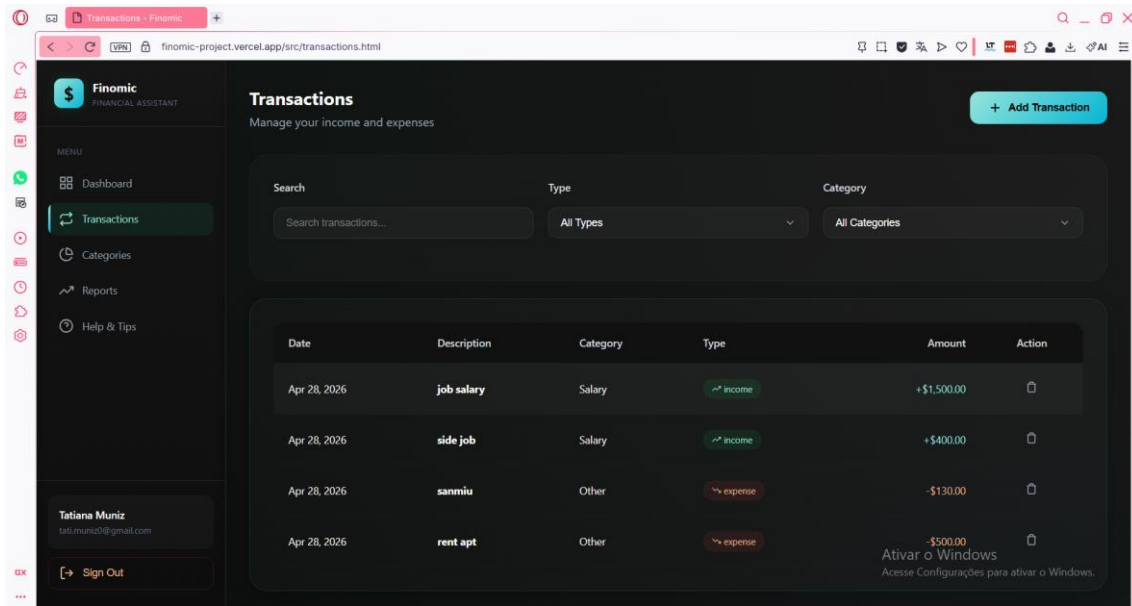
### Step 4 – Dashboard (After Login)



Screenshot 4: The main dashboard showing total balance (\$1,270.00), total income (\$1,900.00), total expenses (\$630), and the calendar.

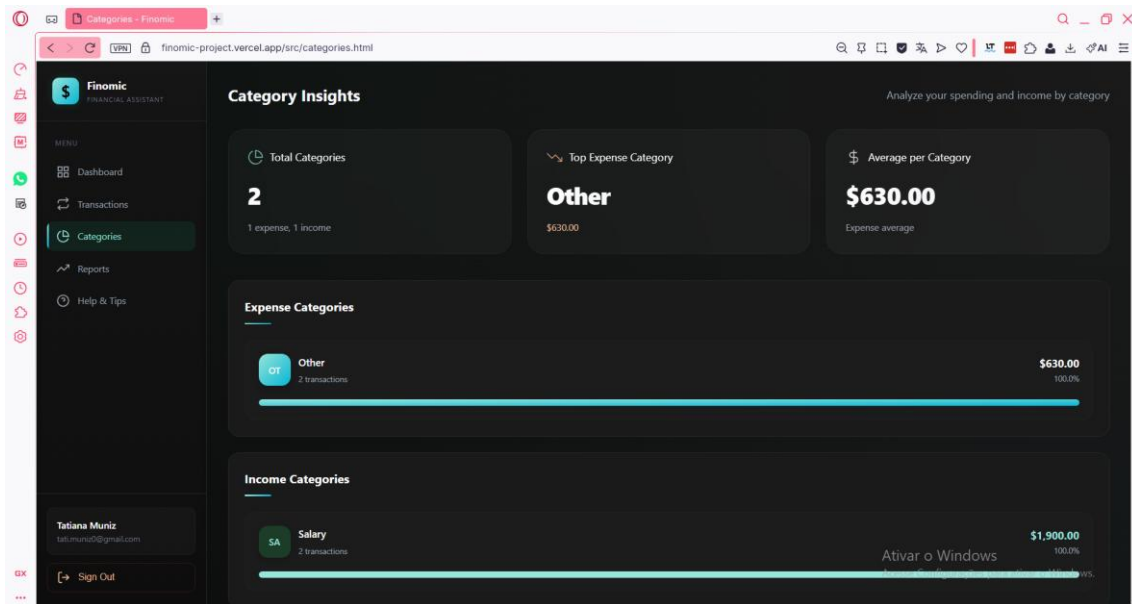
Screenshot 5: The main dashboard showing the overview, financial health, and recent transactions.

## Step 5 – Transactions Page



Screenshot 6: Transaction page showing the add transaction button, and search by string, type, and category, and a table of all the transactions.

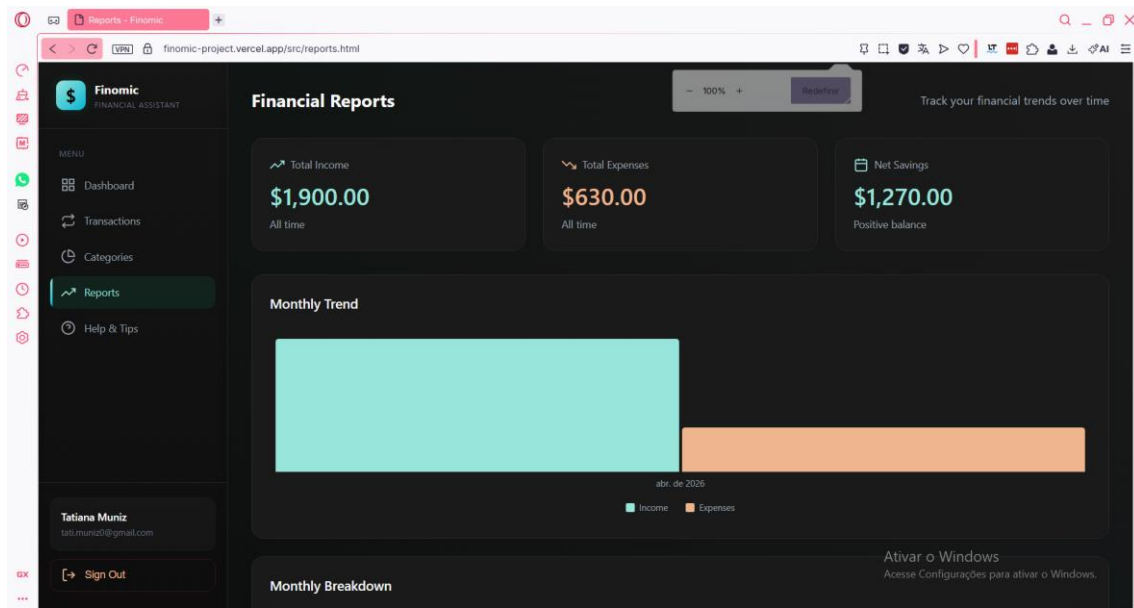
## Step 6 – Categories Page



Screenshot 7: Categories page showing the category insights as the total categories (2), top expense category (Other), and the average per category (\$630). As well as a percentage bar list for the expense and income categories.

## Step 7 – Reports Page





**Monthly Breakdown**

| Month        | Income     | Expenses | Net         | Savings Rate |
|--------------|------------|----------|-------------|--------------|
| abr. de 2026 | \$1,900.00 | \$630.00 | +\$1,270.00 | 66.8%        |

**CISC3003 Web Programming**  
Financial Assistant Dashboard - Group 03

**Team Members**  
Tatiana Muniz Rodríguez (DC027021) • Diogo Barroo (DC328252) • Diana (DC325411) • Elly (DC327168) • Edith (DC328581)

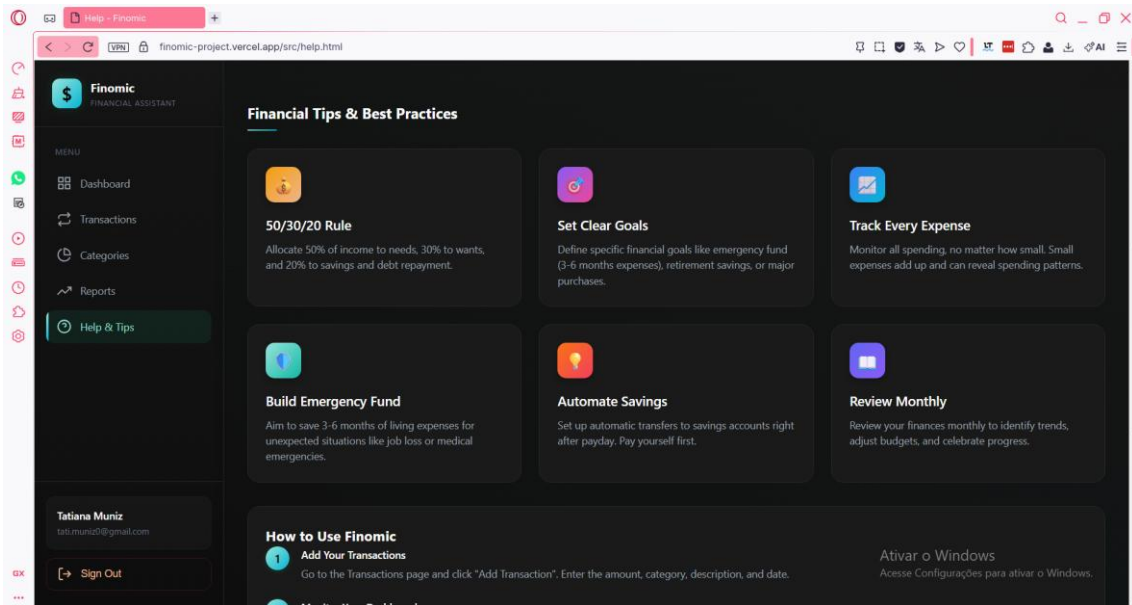
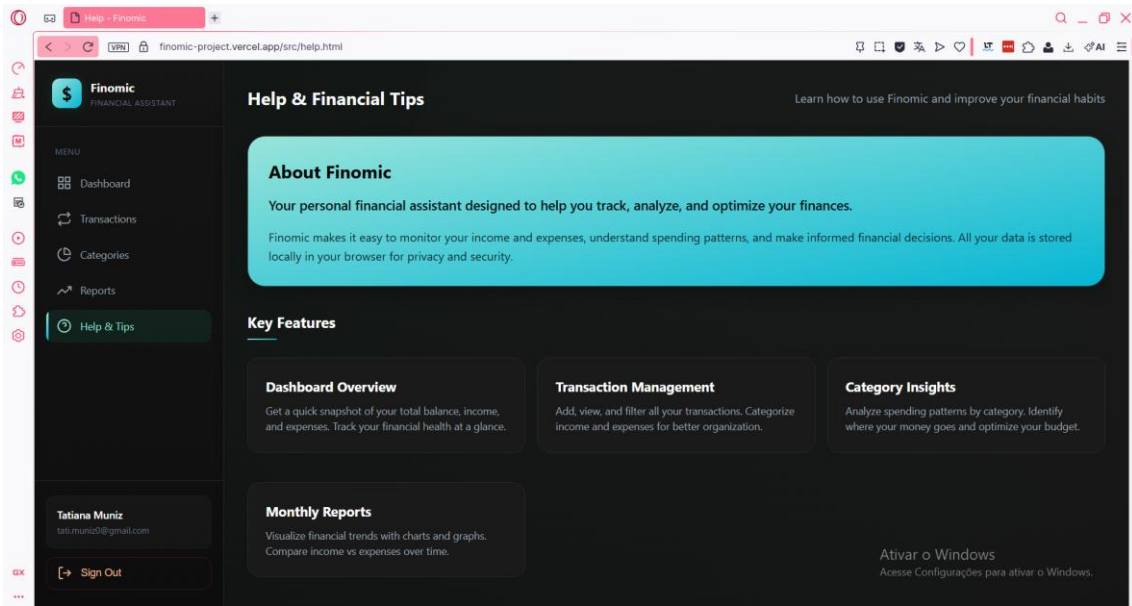
© 2026 Finomic. All rights reserved.

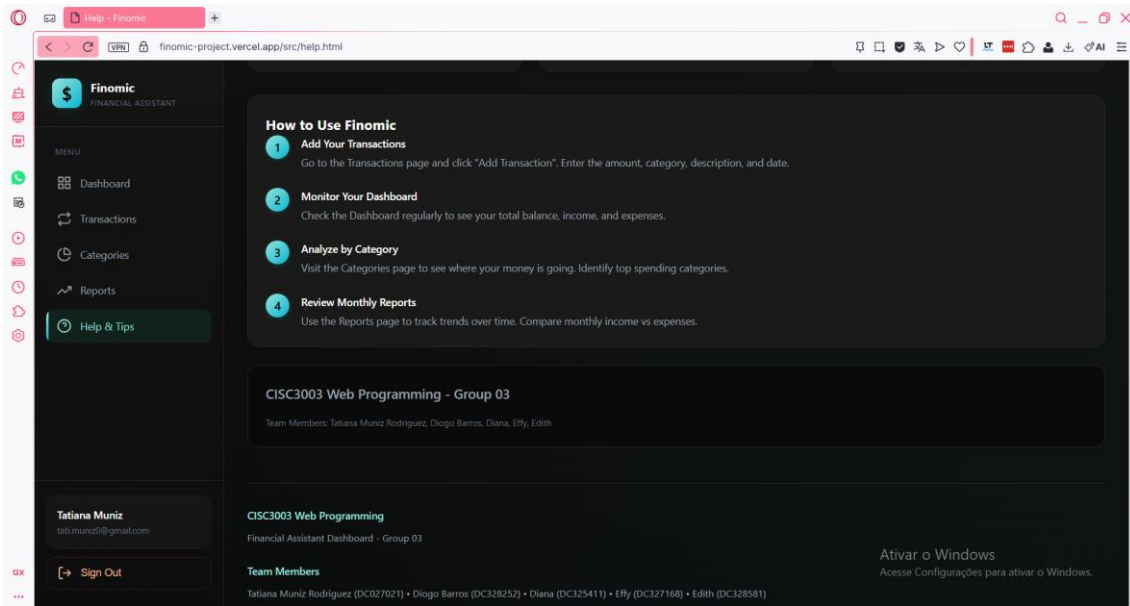
Ativar o Windows  
Acesse Configurações para ativar o Windows.

*Screenshot 8: The reports page with a financial report on total income, total expense and net saving, and a monthly trend bar chart.*

*Screenshot 9: The reports page showing the monthly breakdown.*

## Step 8 – Help Page



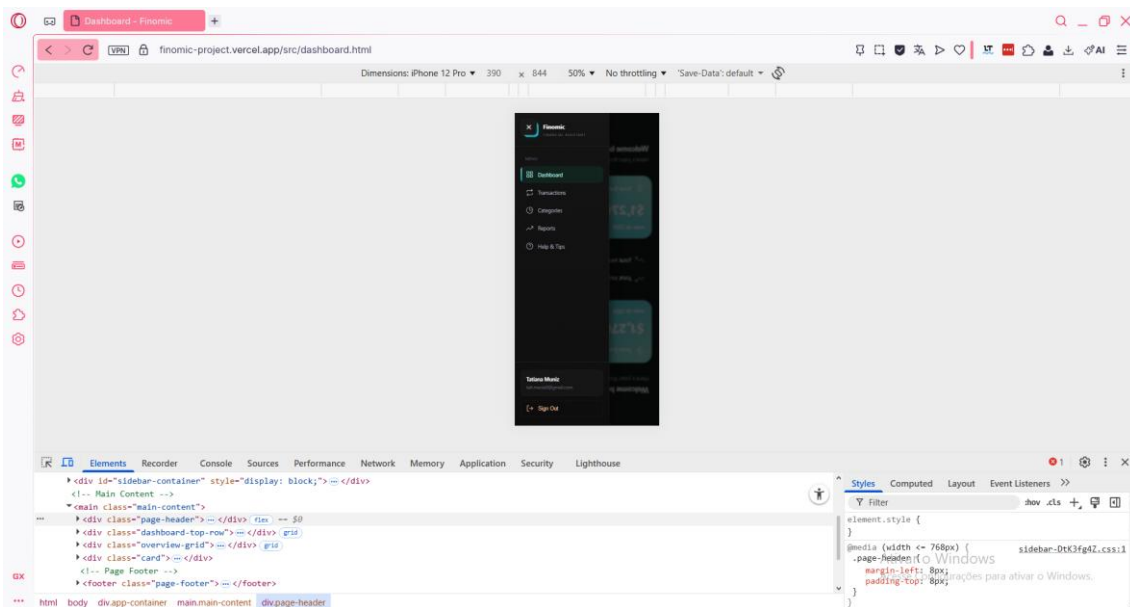


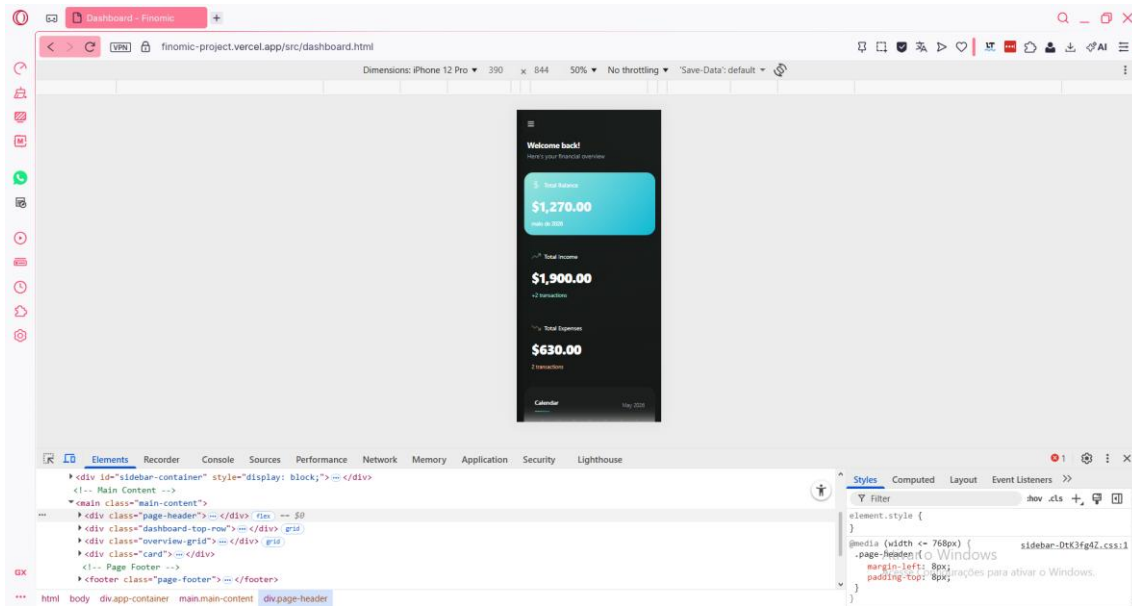
Screenshot 10: The help page showing a sentence about Finomic and explaining the key features of the web service.

Screenshot 11: The help page showing financial tips and best practices.

Screenshot 12: The help page showing a step-by-step on how to use Finomic.

## Step 9 – Responsive Mobile View

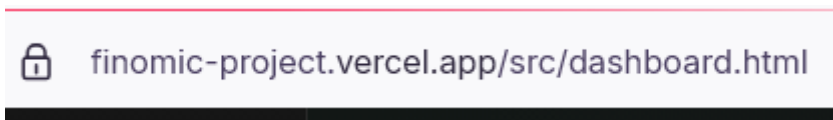




Screenshot 13: Mobile device view (width <768px), showing the open hamburger menu.

Screenshot 14: The dashboard as seen on a mobile device (width <768px).

## Step 10 – Deployment Confirmation



Screenshot 15: The browser tab showing the live deployed URL.

# Installation Instructions

To install and run Finomic locally for development or testing, follow these steps:

## 1. Prerequisites

- Node.js (v16 or higher) and npm installed on your machine.

## 2. Clone the Repository

```
git clone https://github.com/tatimunizz/CISC3003-DC02702-2026-GROUP03-PROJECT.git
```

```
cd CISC3003-DC02702-2026-GROUP03-PROJECT
```

## 3. Install Dependencies

```
npm install
```

(This installs Vite and any other development dependencies.)

#### 4. Configure Environment Variables

- Copy the example environment file:

```
cp .env.example .env
```

- Open `.env` and add your own Firebase project credentials. (Obtain these from the Firebase Console > Project Settings > General > Your Apps.)
- **Important:** Do not commit the `.env` file to version control. It is already ignored via `.gitignore`.

#### 5. Run the Development Server

```
npm run dev
```

#### 6. Access the Application

Open your browser and navigate to <http://localhost:5173> (the exact port may vary; see the terminal output).

#### 7. Build for Production

To create a production build, run:

```
npm run build
```

The built files will be in the `dist/` directory.

## Project Citations and References

Firebase. (n.d.). Firebase documentation. Retrieved from <https://firebase.google.com/docs>

MDN Web Docs. (2024). JavaScript: Dynamic client-side scripting. Mozilla Developer <https://developer.mozilla.org/en-US/docs/Learn/JavaScript>

MDN Web Docs. (2024). CSS: Cascading Style Sheets. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/CSS>

Mozilla Developer Network. (2025). Web technologies for developers. Retrieved from <https://developer.mozilla.org/en-US/docs/Web>

Vite.js Team. (2025). Vite - Next generation frontend tooling. Retrieved from <https://vitejs.dev/>

Google. (n.d.). Material Design 3 guidelines. Retrieved from <https://m3.material.io/>

W3Schools. (2024). Web development tutorials. <https://www.w3schools.com/>

## Project Declaration of AI Usage

This project report was prepared with the assistance of a large language model (LLM). Specifically:

- **Code assistance:** The LLM was used to help learn about the stack being used, as well to interpret the documentation, step-by-step guide on how to set up Firebase Authentication, for example. It was used to help debug JavaScript errors, and solve CSS and HTML issues.
- **Proofreading:** The LLM was used to check for grammar and clarity in the final report.